

week2

2026-09-07

Question 1

a)

```
set.seed(43202)

n <- 100
p <- 20
X <- matrix(rnorm(n * p), n, p)
beta <- 0.4^sqrt(1:p)
mu <- X %*% beta

train <- test <- matrix(0, 200, p + 1)

for (s in 1:200) {
  y <- mu + rnorm(n)
  y_test <- mu + rnorm(n)

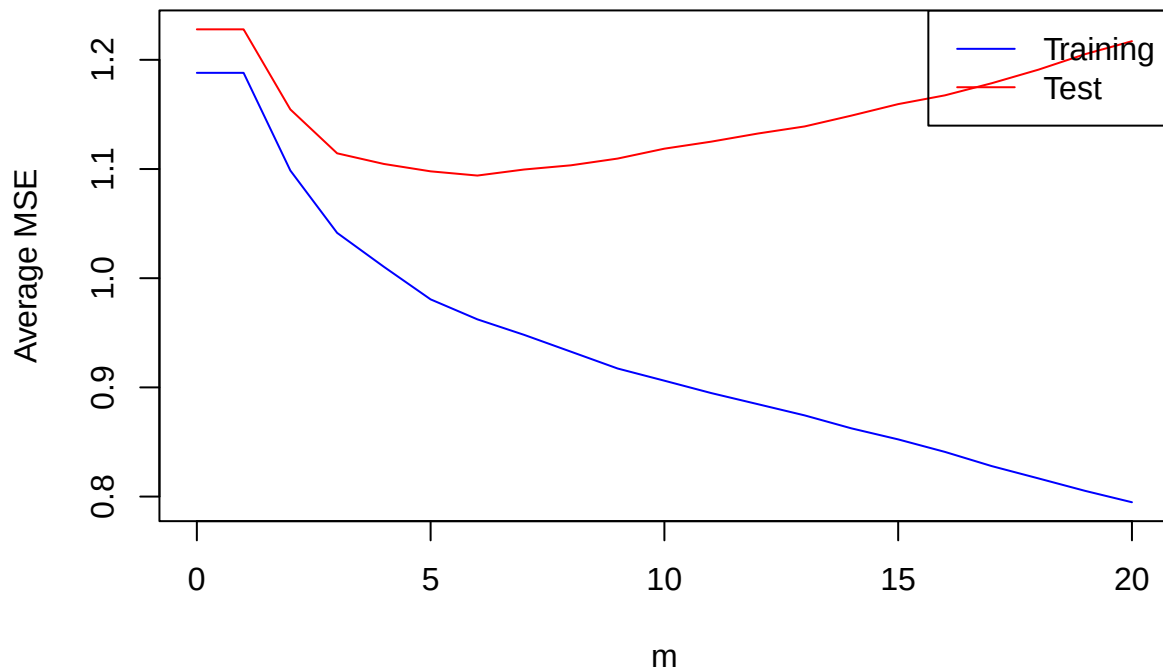
  for (m in 0:p) {
    Xm <- cbind(1, X[, 1:m, drop = FALSE])
    fit <- lm.fit(Xm, y)
    y_hat <- Xm %*% fit$coefficients

    train[s, m + 1] <- mean((y - y_hat)^2)
    test[s, m + 1] <- mean((y_test - y_hat)^2)
  }
}

plot(0:p, colMeans(train), type = "l", col = "blue",
     ylim = range(colMeans(train), colMeans(test)),
     xlab = "m", ylab = "Average MSE")

lines(0:p, colMeans(test), col = "red")

legend("topright", c("Training", "Test"),
     col = c("blue", "red"), lty = 1)
```



```
all(apply(train, 1, function(x) all(diff(x) <= 1e-10)))
```

```
## [1] TRUE
```

The average training MSE decreases as predictors are added, while the average test MSE first decreases and may later increase. The numerical check returned TRUE, confirming that training MSE was non-increasing in every simulation run.

b)

```
bias2 <- numeric(p + 1)

for (m in 0:p) {
  if (m == 0) {
    Xm <- matrix(1, n, 1)
  } else {
    Xm <- cbind(1, X[, 1:m, drop = FALSE])
  }

  mu_hat <- Xm %*% lm.fit(Xm, mu)$coefficients
  bias2[m + 1] <- sum((mu - mu_hat)^2)
}

theory_train <- bias2 / n + 1 - (0:p + 1) / n
theory_test <- bias2 / n + 1 + (0:p + 1) / n
```

The theoretical training and test MSE curves are close to their corresponding simulation averages.

c)

```
plot(
  0:p,
  colMeans(train),
  type = "l",
  col = "blue",
  lwd = 2,
  ylim = range(
    train[1, ],
    colMeans(train),
    colMeans(test),
    theory_train,
    theory_test
  ),
  xlab = "m",
  ylab = "MSE"
)

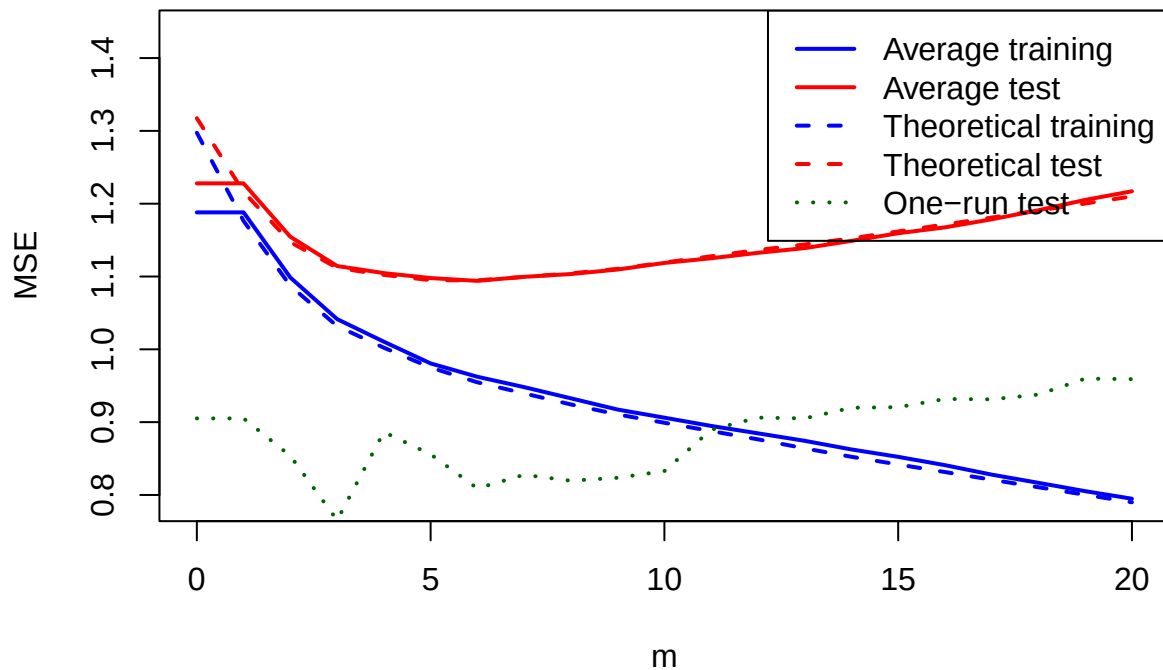
lines(0:p, colMeans(test), col = "red", lwd = 2)

lines(0:p, theory_train,
      col = "blue", lty = 2, lwd = 2)

lines(0:p, theory_test,
      col = "red", lty = 2, lwd = 2)

# Test MSE from the first simulation run
lines(0:p, test[1, ],
      col = "darkgreen", lty = 3, lwd = 2)

legend(
  "topright",
  c(
    "Average training",
    "Average test",
    "Theoretical training",
    "Theoretical test",
    "One-run test"
  ),
  col = c("blue", "red", "blue", "red", "darkgreen"),
  lty = c(1, 1, 2, 2, 3),
  lwd = 2
)
```



The test MSE curve from one simulation run is less smooth than the average test MSE curve because it depends on the particular random training and test errors generated in that run. Averaging over 200 simulations reduces these random fluctuations and produces a curve closer to the expected test MSE.

Question 2

```
n <- 100
p <- 6
X_all <- outer(
  1:n, 1:p,
  function(i, j) sqrt(2) * cos(pi * j * (i - 0.5) / n)
)
```

a)

```
n <- 100
sigma2 <- 1
p <- 6

beta <- rep(0.5, p)
x0 <- c(0, 1, 0, 0, 1, 0)

mu0 <- sum(x0 * beta)

bias2 <- expected_error <- numeric(p + 1)

for (m in 0:p) {
```

```

omitted <- if (m == p) 0 else sum(x0[(m + 1):p] * beta[(m + 1):p])

bias2[m + 1] <- omitted^2
expected_error[m + 1] <-
  bias2[m + 1] + sigma2 / n * (1 + sum(x0[seq_len(m)]^2))
}

data.frame(
  m = 0:p,
  squared_bias = bias2,
  expected_error = expected_error
)

```

```

##   m squared_bias expected_error
## 1 0          1.00          1.01
## 2 1          1.00          1.01
## 3 2          0.25          0.27
## 4 3          0.25          0.27
## 5 4          0.25          0.27
## 6 5          0.00          0.03
## 7 6          0.00          0.03

```

The true mean response at x_0 is $\mu_{x_0} = x_0^T \beta = 1$. The expected squared prediction error consists of a squared bias term caused by omitted predictors and a variance term caused by estimating the intercept and included coefficients.

b)

```

X_all <- outer(
  1:n, 1:p,
  function(i, j) sqrt(2) * cos(pi * j * (i - 0.5) / n)
)

set.seed(43202)

sim_error <- matrix(0, nrow = 200, ncol = p + 1)

for (s in 1:200) {
  y <- X_all %*% beta + rnorm(n)

  for (m in 0:p) {
    Xm <- cbind(1, X_all[, seq_len(m), drop = FALSE])
    fit <- lm.fit(Xm, y)

    x0m <- c(1, x0[seq_len(m)])
    mu0_hat <- sum(x0m * fit$coefficients)

    sim_error[s, m + 1] <- (mu0_hat - mu0)^2
  }
}

average_sim_error <- colMeans(sim_error)

```

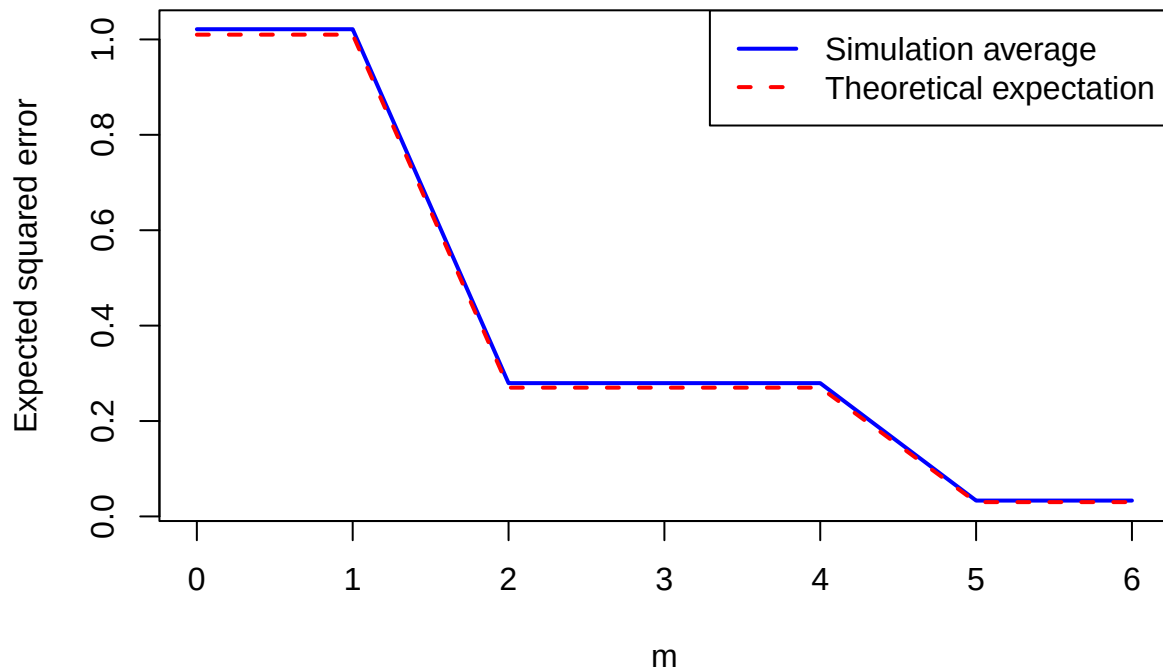
```

plot(
  0:p, average_sim_error,
  type = "l",
  col = "blue",
  lwd = 2,
  ylim = range(average_sim_error, expected_error),
  xlab = "m",
  ylab = "Expected squared error"
)

lines(
  0:p, expected_error,
  col = "red",
  lty = 2,
  lwd = 2
)

legend(
  "topright",
  c("Simulation average", "Theoretical expectation"),
  col = c("blue", "red"),
  lty = c(1, 2),
  lwd = 2
)

```



The simulation averages are close to the theoretical expected squared errors, with small differences due to

random simulation variation. The minimum theoretical expected squared error is 0.03, attained at both $m=5$ and $m=6$. Therefore, the smallest predictor count attaining the minimum is $\boxed{m = 5}$

c)

Although all six regression coefficients are nonzero, only predictors 2 and 5 contribute directly to the mean response at x_0 . This is because

$$x_0 = (0, 1, 0, 0, 1, 0)^\top,$$

so only x_{02} and x_{05} are nonzero. Therefore,

$$\mu_0 = x_0^\top \beta = \beta_2 + \beta_5 = 0.5 + 0.5 = 1.$$

The squared prediction error in this problem is point-specific because it measures prediction performance only at x_0 . A predictor with a zero coordinate at x_0 does not directly affect the mean response at this point, even if its regression coefficient is nonzero.

In contrast, the test MSE in the previous problem averages the squared prediction errors over all 100 rows of X_{all} . Since all predictors vary across these observations, all six predictors can affect the overall test MSE.

Question 3

The expected training and test MSEs are

$$E(\text{MSE}_{\text{train}} \mid X) = \frac{B^2}{n} + \sigma^2 \left(1 - \frac{p+1}{n} \right),$$

and

$$E(\text{MSE}_{\text{test}} \mid X) = \frac{B^2}{n} + \sigma^2 \left(1 + \frac{p+1}{n} \right).$$

Therefore,

$$E(\text{MSE}_{\text{test}} - \text{MSE}_{\text{train}} \mid X) = \frac{2(p+1)\sigma^2}{n}.$$

Using $n = 120$, $p = 4$, $\sigma^2 = 1$, and $B^2/n = 0.04$,

$$E(\text{MSE}_{\text{train}} \mid X) = 0.04 + 1 - \frac{5}{120} = 0.998333,$$

and

$$E(\text{MSE}_{\text{test}} \mid X) = 0.04 + 1 + \frac{5}{120} = 1.081667.$$

Thus, the expected difference is 0.083333. These are long-run expectations, so a particular test MSE may still be smaller than its corresponding training MSE.

The corrected estimate of test MSE is

$$\widehat{\text{MSE}}_{\text{test}} = \frac{116.4 + 2(5)(0.96)}{120} = 1.05.$$

Mallows' C_p is

$$C_p = \frac{116.4}{0.96} - 120 + 2(5) = 11.25.$$

The two criteria are related by

$$\frac{\hat{\sigma}^2(C_p + n)}{n} = \frac{0.96(11.25 + 120)}{120} = 1.05.$$

Since n and $\hat{\sigma}^2$ are the same for all candidate models, this is an increasing transformation. Therefore, the corrected test MSE and Mallows' C_p rank the models in the same order.

Question 4

a)

```
d <- read.csv("data/diabetes.csv")[1:370, ]
n <- 370

fit_A <- lm(y ~ bmi + bp + s5 + sex + s1 + s2 + s4, data = d)
fit_B <- lm(y ~ bmi + bp + s5 + sex + s1 + s2 + s4 + s6, data = d)
fit_full <- lm(y ~ age + sex + bmi + bp + s1 + s2 + s3 +
              s4 + s5 + s6, data = d)

RSS <- c(
  A = deviance(fit_A),
  B = deviance(fit_B),
  Full = deviance(fit_full)
)

full_df <- df.residual(fit_full)
sigma2_hat <- RSS["Full"] / full_df

data.frame(
  Model = c("A", "B", "Full"),
  p = c(7, 8, 10),
  RSS = RSS
)
```

```
##      Model  p    RSS
## A         A  7 1098909
## B         B  8 1089717
## Full    Full 10 1089452
```

```
full_df
```

```
## [1] 359
```

```
sigma2_hat
```

```
##      Full
## 3034.684
```

b)

```
criterion <- function(RSS, p) {  
  c(  
    Cp = RSS / sigma2_hat - n + 2 * (p + 1),  
    AIC_red = n * log(RSS / n) + 2 * (p + 1),  
    BIC_red = n * log(RSS / n) + (p + 1) * log(n)  
  )  
}  
  
results <- rbind(  
  A = criterion(RSS["A"], 7),  
  B = criterion(RSS["B"], 8)  
)  
  
results
```

```
##      Cp.A AIC_red.A BIC_red.A  
## A 8.116253 2974.640 3005.948  
## B 7.087434 2973.532 3008.754
```

c)

```
RSS_improvement <- RSS["A"] - RSS["B"]  
  
c(  
  RSS_improvement = RSS_improvement,  
  Cp_penalty = 2 * sigma2_hat,  
  AIC_fit_improvement = n * log(RSS["A"] / RSS["B"]),  
  AIC_penalty = 2,  
  BIC_penalty = log(n)  
)
```

```
##      RSS_improvement.A      Cp_penalty.Full AIC_fit_improvement.A  
##      9191.508850          6069.368920          3.107775  
##      AIC_penalty          BIC_penalty  
##      2.000000          5.913503
```

Adding s_6 reduces RSS by approximately 9191.51. For C_p , this improvement is greater than the one-parameter penalty $2\hat{\sigma}^2 = 6069.37$. For reduced AIC, the improvement in fit is 3.108, which is greater than its penalty of 2. Therefore, both C_p and AIC select Model B. In contrast, BIC uses a larger penalty of $\log(370) = 5.914$, so it selects Model A.

Selecting a model does not prove that it is the true data-generating model because these criteria only compare the given candidate models using one finite dataset. Their selections depend on different tradeoffs between goodness of fit and model complexity.

Question 5

a)

The first misuse happens in Step 3, when the analyst uses the final test data to select the model. The smallest test MSE is usually too low because one model may perform well partly because of random chance.

b)

Split the training data into five folds. For each model, train it on four folds and test it on the remaining fold. Repeat this five times and calculate the average cross-validation MSE. Select the number of predictors with the smallest average MSE. Then refit that model using all 370 training observations. Finally, use rows 371–442 once to calculate the final test MSE.

c)

The analyst can report all eleven test MSE values and clearly state that the test data were used for model selection. However, the smallest MSE should not be treated as a fair estimate of future prediction error. A new independent test dataset is needed for a new final evaluation.